Limitations and Next Steps

Current Limitations I

GNN 5.0.3 is a working system with deliberately scoped boundaries, and it is worth stating those boundaries plainly rather than implying broader completeness than the artifacts support. The reliability gates that underpin our confidence in the system — the semantic-fidelity ledger and the cross-framework reliability ledger — are reproducible by command and recorded against the 9 model families, but this manuscript does not assert a particular full-suite pass rate as a headline number. Pass and skip counts shift with the local toolchain, the presence or absence of optional simulation dependencies, and whether the optional Ollama integration paths are exercised, so we retain the reproducing command together with its source, environment, selected-input, and artifact bindings, leaving run-specific tallies in the ledgers and execution traces. A fresh run supplies new evidence for its own environment; it does not replace a preserved historical receipt.

Current Limitations I

A second limitation lives at the boundary between the model families and the rendering backends. GNN registers 12 simulation backends, but coverage across the family-by-backend matrix is intentionally uneven, and the gaps are recorded as explicit *profiled-unsupported* statuses rather than silently omitted. The cross-framework acceptance check is anchored on the continuous family, where JAX, NumPyro, RxInfer.jl are compared directly; selected continuous and hierarchical lanes carry profiled-unsupported markers at the render and execute steps (Steps 11 and 12) for backends that cannot yet faithfully realize their continuous-state or temporal-depth structure. This is honest bookkeeping, not a defect to be hidden: an unsupported status with a recorded reason is more trustworthy than a forced translation that quietly misrepresents the model. It also reflects the state of the surrounding literature, where expected-free-energy formulations,

Current Limitations I

variational-inference reductions, and hierarchical active-inference methods are still being actively refined rather than collapsed into one settled executable recipe (Champion et al. 2024; Nuijten et al. 2026; Rangarajan and Rao 2026). The practical consequence is that not every model expressible in the GNN syntax can be executed on every registered backend today, and authors should consult the profiled-unsupported ledger that `scripts/run_model_family_acceptance.py` writes — the command is given in sec. 6 — before assuming a given family will round-trip through a given framework. The per-kind shape of that matrix matters as much as its gaps: the categorical-only backends report the continuous linear-Gaussian kind `unsupported` by declared capability rather than by accident, structural wrappers have no renderable model body and are reported `unsupported`, and the recursive corpus is reserved for bounded

Current Limitations I

proposal-only `--autonomous` runs and holds no committed models yet — boundaries recorded where the kinds are defined (sec. 3.2) rather than discovered mid-run.

A third limitation is one of scope rather than capability. The formal account in sec. 3 fixes each kind's parameterization as given — the categorical tensors `A`, `B`, `C`, `D`, and `E` of the discrete kind and the system matrices, Gaussian prior, and optional control declarations of the linear-Gaussian kind — and states only belief-level inference: state and policy inference for the categorical kind (eq. 6, eq. 7) and fixed-parameter filtering and control for the continuous kind. GNN can also express *parameter* learning: a matrix may be declared as a latent variable with a Dirichlet prior, and the repository ships both a corpus model that does so (`input/gnn_files/learning/dirichlet_likelihood_learning.md`,

Current Limitations I

which learns its likelihood matrix `A` from observations by joint variational message passing) and the renderer strategy that emits code for it (`src/gnn/render/rxinfer/_strategies_learning.py`). That corpus directory is registered in no manifest family — the unregistered set is exactly `input/gnn_files/learning`, `input/gnn_files/recursive`, `input/gnn_files/thrml` — so parameter learning is exercised by neither the semantic-fidelity nor the cross-framework ledger and carries no figure or table in this manuscript. We state it here as implemented-but-out-of-scope rather than leave the omission to be inferred from the family tables.

Current Limitations I

Experimental targets also have explicit admission boundaries. THRML performs fixed-action categorical posterior sampling with bounded independent or validated joint composition; it does not supply Gaussian inference or policy/EFE optimization. cpomdp accepts linear-Gaussian models with resource-admitted finite policy search, and rejects unsupported kinds or over-budget search rather than reducing the request silently. Both require explicit framework selection, so registration does not promote either into the default execution set.

Current Limitations I

Finally, several capabilities depend on optional dependencies that are not part of the minimal install. The simulation frameworks themselves, the audio sonification path, the LLM-enhanced analysis step, and the interactive visualization step all require extra packages that a lean checkout will not have, and readiness diagnoses distinguish absent packages, toolchains, unsupported Python, failed probes, and timeouts. Their recorded outcomes depend on the step's requirements; missing dependencies or required unfinished work do not establish successful execution. This keeps the core parse-validate-export-visualize spine runnable in constrained environments, but it means a full 0–24 traversal of all 25 steps reflects the optional surface that a particular machine has installed. We consider this an acceptable engineering trade for portability, but it is a limitation a reader should hold in mind when interpreting any single end-to-end run.

Next Steps I

The roadmap is concrete and staged. The current software version is GNN 5.0.3. It builds on the three safe-by-design orchestration contracts introduced in GNN 3.0.0, which generate and validate data only, with no live infrastructure mutation. The first is *durable observation streams*: file- and array-backed stream manifests (content-checksummed) with replayable execution traces, so that an extended run can be observed, paused, and re-derived deterministically before any live sensor or device-backed stream is ever introduced. The second is *resumable run sessions*: run-session manifests with atomic checkpoint/resume, status inspection, and cancellation-safe cleanup so that extended model-family acceptance runs can be interrupted and resumed without corrupting partial state. The third is *auditable container plans*: generating hardened container plans with an explicit static security-review and rollback descriptor, deliberately stopping

Next Steps I

short of mutating any real cluster. Each contract is covered by tests over real objects with negative controls and a strict acceptance gate, additive live wiring connects them to the session-acceptance and run-manifest paths, and three new Model Context Protocol tools expose them. The unifying discipline across all three is that orchestration becomes more capable only as fast as it becomes more inspectable.

The next major release pushes toward bounded autonomy, and it is gated behind the 3.0.0 orchestration work for good reason. The intent is to promote today's proposal-only candidate-scoring machinery — which already ranks candidate patches using the existing validators, model-family ledgers, and interpretability reports — toward *reviewed self-editing of GNN files*. Crucially, the design keeps a human in the loop: edits are proposed and applied only after explicit user approval, with no automatic source mutation, and the autonomy is

Next Steps I

wrapped in policy, rollback, and audit controls before any self-modifying or distributed action is permitted. This continues the same principle that governs the rest of the system, in the lineage of active-inference accounts of self-organizing systems that act to minimize free energy under explicit generative models (Friston 2010; Parr et al. 2022): an agent should expand its license to act only in proportion to the reviewability of what it does. The next-release items are not claimed as complete here; they are the recorded direction of travel, and each will be evidenced by its own gates when it lands, just as the 3.0.0 contracts are evidenced by theirs today. On the model-kind axis the same staging applies: extending native hierarchical rendering beyond the two-level strategies, broadening continuous reach across the remaining registered backends, and populating the reserved recursive corpus each arrive with their own gate evidence rather than by prose

Next Steps I

assertion.